Game Theory-Assignment6

Xinyi Yang

February 23rd 2026

Exercise 13

Each cell of the Sudoku grid is a player. Player i selects a digit and belongs to:

$$r_i \in \{1, \ldots, 9\}, \quad c_i \in \{1, \ldots, 9\}, \quad b_i \in \{1, \ldots, 9\}.$$

Define

- $\Delta_r^{row}(p)$ = number of repeated digits in row r
- $\Delta_c^{col}(p)$ = number of repeated digits in column c
- $\Delta_b^{block}(p)$ = number of repeated digits in block b

Player i's cost:

$$J_i(p) = \Delta_{r_i}^{row}(p) + \Delta_{c_i}^{col}(p) + \Delta_{b_i}^{block}(p).$$

Potential Function

Define

$$\Phi(p) = \frac{1}{9} \sum_{i=1}^{81} J_i(p).$$

Since each row has 9 players,

$$\sum_{i=1}^{81} \Delta_{r_i}^{row}(p) = 9 \sum_{r=1}^{9} \Delta_r^{row}(p).$$

Similarly for columns and blocks.
Therefore

$$\Phi(p) = \sum_{r=1}^{9} \Delta_r^{row}(p) + \sum_{c=1}^{9} \Delta_c^{col}(p) + \sum_{b=1}^{9} \Delta_b^{block}(p).$$

Exact Potential Property

If player i changes action, only:

$$r_i, \ c_i, \ b_i$$

can change.
Thus

$$J_i(p_i', p_{-i}) - J_i(p) = \Delta_{r_i}^{row} + \Delta_{c_i}^{col} + \Delta_{b_i}^{block} \text{ (change)}.$$

All other rows, columns, blocks unchanged.
Hence

$$\Phi(p_i', p_{-i}) - \Phi(p) = J_i(p_i', p_{-i}) - J_i(p).$$

Therefore the game is an exact potential game.

$$\Phi(p) = \sum_r \Delta_r^{row} + \sum_c \Delta_c^{col} + \sum_b \Delta_b^{block}$$

proved.

Exercise 14

MATLAB Implementation

```
clear; clc; rng('shuffle');

board = [5 3 0  0 7 0  0 0 0;
         6 0 0  1 9 5  0 0 0;
         0 9 8  0 0 0  0 6 0;
         8 0 9  0 6 0  0 0 3;
         4 0 0  8 5 3  0 0 1;
         7 0 0  9 2 0  0 5 6;
         0 6 0  0 0 0  2 8 0;
         0 0 0  1 9 0  0 5 0;
         0 0 0  0 8 0  0 7 9];

actionSpaces = sudoku_actionspaces(board);
nRepeats = 1000;
potentials = zeros(nRepeats, 1);

for k = 1:nRepeats
    A0 = nan(size(actionSpaces));
    for i = 1:numel(actionSpaces)
        avail = actionSpaces{i};
        A0(i) = avail(randi(length(avail)));
    end

    [A, J] = improvementPath(actionSpaces, A0);

    potentials(k) = sum(J(:)) / 9;
end

figure;
histogram(potentials, 'BinWidth', 1, 'FaceColor', 'b', 'EdgeColor', 'k');
xlabel('Potential function');
ylabel('frequency');
title(sprintf('Histogram of the Nash equilibrium potential function obtained from 1000 runs
    (zero potential proportion: %.2f%%)', ...
              sum(potentials == 0)/nRepeats*100));
grid on;

function actionSpaces = sudoku_actionspaces(board)
    actionSpaces = cell(size(board));
    for row = 1:9
        for col = 1:9
            if board(row,col) ~= 0
                actionSpaces{row,col} = board(row,col);
            else
                actionSpaces{row,col} = 1:9;
                actionSpaces{row,col} = setdiff(actionSpaces{row,col}, board(row,:));
                actionSpaces{row,col} = setdiff(actionSpaces{row,col}, board(:,col));
                rowBlk = 3*floor((row-1)/3) + (1:3);
                colBlk = 3*floor((col-1)/3) + (1:3);
```

```
                actionSpaces{row,col} = setdiff(actionSpaces{row,col}, board(rowBlk,colBlk))
    ;
            end
        end
    end
end

function J = funJ(A)
    J = zeros(size(A));
    for row = 1:9
        for col = 1:9
            rowBlk = 3*floor((row-1)/3) + (1:3);
            colBlk = 3*floor((col-1)/3) + (1:3);
            J(row,col) = sum(A(row,:) == A(row,col)) + ...
                         sum(A(:,col) == A(row,col)) + ...
                         sum(sum(A(rowBlk,colBlk) == A(row,col))) - 3;
        end
    end
end

function [A, J] = improvementPath(actionSpaces, A0)
    A = A0;
    [nRows, nCols] = size(actionSpaces);
    N = numel(actionSpaces);
    improved = true;
    maxIter = 1000;
    iter = 0;
    while improved && iter < maxIter
        improved = false;
        iter = iter + 1;
        order = randperm(N);
        for idx = order
            [r,c] = ind2sub([nRows,nCols], idx);
            oldVal = A(r,c);
            bestVal = oldVal;
            bestCost = inf;
            currentCost = computeCost(A, r, c);
            for newVal = actionSpaces{r,c}
                if newVal == oldVal, continue; end
                A(r,c) = newVal;
                newCost = computeCost(A, r, c);
                if newCost < bestCost
                    bestCost = newCost;
                    bestVal = newVal;
                end
            end
            A(r,c) = bestVal;
            if bestVal ~= oldVal
                improved = true;
            end
        end
    end
    J = funJ(A);
end

function cost = computeCost(A, r, c)
    rowBlk = 3*floor((r-1)/3) + (1:3);
    colBlk = 3*floor((c-1)/3) + (1:3);
    cost = sum(A(r,:) == A(r,c)) + sum(A(:,c) == A(r,c)) + ...
           sum(sum(A(rowBlk,colBlk) == A(r,c))) - 3;
end
```

Game-Theoretic Formulation

The Sudoku puzzle is modeled as a multi-player potential game, where each cell of the grid represents a player whose action is the digit placed in that cell.

Each player's cost equals the number of violations of Sudoku constraints, i.e., the number of identical digits appearing in the same row, column, or 3×3 block (excluding the cell itself).

The potential function is the sum of all players' costs. A valid Sudoku solution corresponds to a Nash equilibrium with zero potential.

To search for equilibria, we apply improvement dynamics. Starting from random feasible assignments, players repeatedly switch to actions that reduce their individual costs until no further unilateral improvement is possible.

Numerical Result

The improvement path algorithm was executed 1000 times from random initial configurations.

The histogram of the potential values at the resulting Nash equilibria is shown in Figure 1.

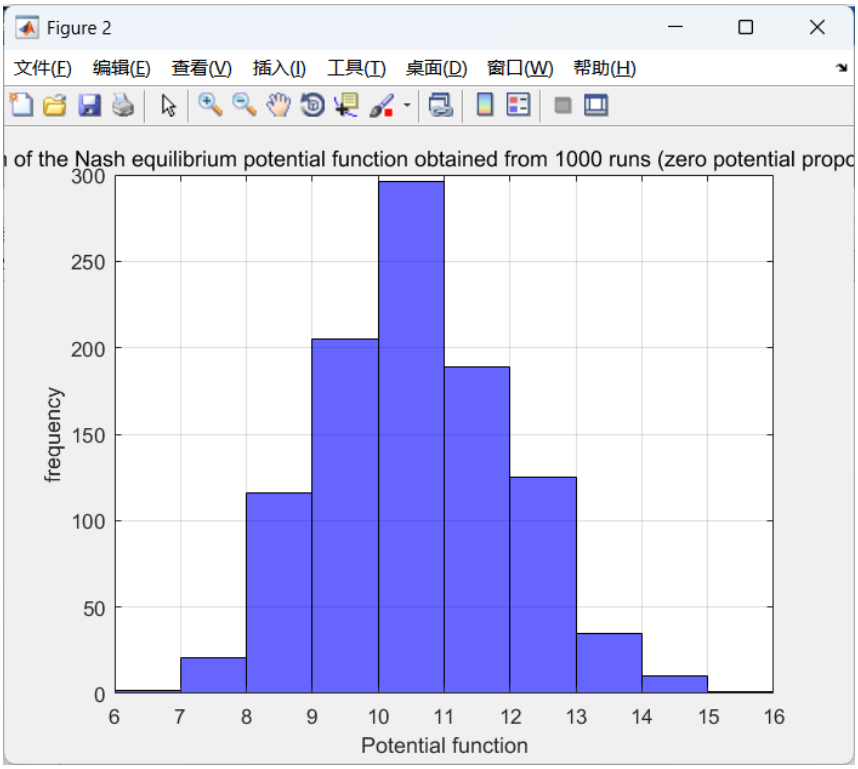

Figure 1: Histogram of potential values obtained from 1000 improvement paths.

The potential values are concentrated around the range 9 to 12. No zero-potential equilibrium was observed among the 1000 runs.

Interpretation

The results show that the improvement dynamics consistently converge to Nash equilibria, but these equilibria typically correspond to local minima of the potential function rather than the global minimum.

Since a valid Sudoku solution requires zero potential, the experiment indicates that such global optima are relatively rare. Most runs become trapped in locally optimal configurations that still violate some Sudoku constraints.

This behavior confirms the theoretical discussion of improvement paths in potential games: convergence to equilibrium is guaranteed, but convergence to the global minimum is not.

Therefore, many independent runs with random initial conditions may be required to obtain a valid Sudoku solution.

Homework #6

Student name :

Exercise	weight	grade (0..4)
13	30%	4
14	70%	3

Grades

4 – correct

3 – mostly correct

2 – half-and-half

1 – mostly incorrect

0 – not done